

保障容器建構安全

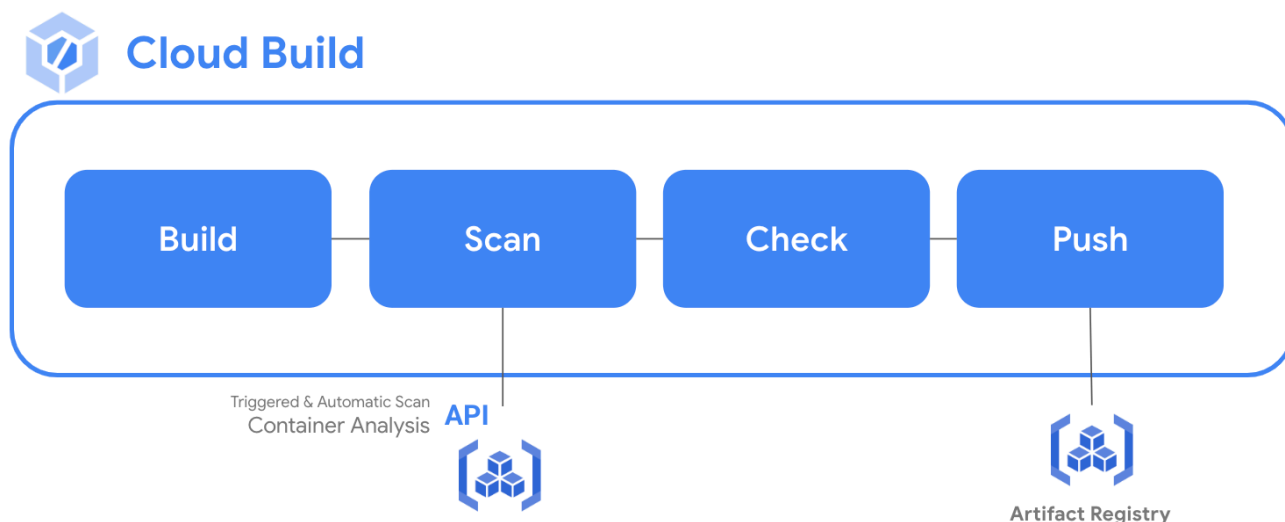
來源：<https://codelabs.developers.google.com/securing-container-builds?hl=zh-tw>

步驟數：8

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [使用 Cloud Build 建構映像檔](#)
4. [Artifact Registry 管理容器](#)
5. [自動掃描安全漏洞](#)
6. [On Demand Scanning](#)
7. [在 Cloud Build 的 CICD 中掃描](#)
8. [恭喜！](#)

1. 簡介



軟體安全漏洞是指可能導致系統意外故障的弱點，或是惡意行為人用來入侵軟體的方法。容器分析提供兩種 OS 掃描功能，可找出容器中的安全漏洞：

- On-Demand Scanning API 可讓您手動掃描容器映像檔，找出 OS 安全漏洞，無論是電腦本機或遠端 Container Registry/Artifact Registry 中的映像檔都適用。
- Container Scanning API 可讓您自動偵測 OS 安全漏洞，每次將映像檔推送至 Container Registry 或 Artifact Registry 時都會掃描。啟用這項 API 後，系統也會進行語言套件掃描，檢查 Go 和 Java 安全漏洞。

On-Demand Scanning API 可讓您掃描儲存在電腦本機的映像檔，或遠端掃描 Container Registry 或 Artifact Registry 中的映像檔。如此一來便能精確控管要掃描哪些容器的安全漏洞。您可以使用 On-Demand Scanning 掃描 CI/CD 管道中的映像檔，再決定是否要儲存在註冊資料庫中。

課程內容

在本實驗室中，您將：

- 使用 Cloud Build 建構映像檔
- 使用 Artifact Registry 管理容器
- 使用自動安全漏洞掃描功能
- 設定 On Demand Scanning
- 在 Cloud Build 的 CICD 中新增映像檔掃描功能

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button, which is circled in blue. Below this, the 'Select a project' section includes a search bar. The 'New Project' section is below that, featuring a 'Project name' field with the text 'my-kubernetes-codelab' and a question mark icon. The 'Project ID' is displayed as 'my-kubernetes-codelab' with a note stating it cannot be changed later. The 'Location' is set to 'No organization'. At the bottom of the 'New Project' section are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

環境設定

在 Cloud Shell 設定專案的專案 ID 和專案編號，分別儲存為 `PROJECT_ID` 和 `PROJECT_ID` 變數。

□□□

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID \
  --format='value(projectNumber)')
```

啟用服務

啟用所有必要服務：

```
gcloud services enable \
  cloudkms.googleapis.com \
  cloudbuild.googleapis.com \
  container.googleapis.com \
  containerregistry.googleapis.com \
  artifactregistry.googleapis.com \
  containerscanning.googleapis.com \
  ondemandscanning.googleapis.com \
  binaryauthorization.googleapis.com
```

3. 使用 Cloud Build 建構映像檔

在本節中，您將建立自動化的建構管道，以打造容器映像檔、執行掃描，然後評估結果。如果未發現重大安全漏洞，系統就會將映像檔推送至存放區。如果發現重大安全漏洞，建構作業就會失敗並結束。

提供 Cloud Build 服務帳戶的存取權

Cloud Build 必須具備 On-Demand Scanning API 的存取權。使用下列指令提供存取權。

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/iam.serviceAccountUser"

gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/ondemandscanning.admin"
```

建立並變更為工作目錄

```
mkdir vuln-scan && cd vuln-scan
```

定義範例圖片

建立名為 Dockerfile 的檔案，其中含有下列內容。

```
cat > ./Dockerfile << EOF
FROM gcr.io/google-
appengine/debian9@sha256:ebffcf0df9aa33f342c4e1d4c8428b784fc571cdf6fbab0b31330347ca8af97a

# System
RUN apt update && apt install python3-pip -y

# App
WORKDIR /app
COPY . ./

RUN pip3 install Flask==1.1.4
RUN pip3 install gunicorn==20.1.0

CMD exec gunicorn --bind :$PORT --workers 1 --threads 8 --timeout 0 main:app

EOF
```

建立名為 main.py 的檔案，其中含有下列內容：

```
cat > ./main.py << EOF
import os
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello_world():
    name = os.environ.get("NAME", "Worlds")
    return "Hello {}!".format(name)

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=int(os.environ.get("PORT", 8080)))
EOF
```

建立 Cloud Build 管道

下列指令會在目錄中建立 `cloudbuild.yaml` 檔案，供自動化程序使用。本範例的步驟僅限於容器建構程序。不過，實際上，除了容器步驟，您還會加入應用程式專用操作說明和測試。

使用下列指令建立檔案。

```
cat > ./cloudbuild.yaml << EOF
steps:

# build
- id: "build"
  name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-
repo/sample-image', '.']
  waitFor: ['-']

EOF
```

執行 CI 管道

提交建構作業以供處理

```
gcloud builds submit
```

查看建構作業詳細資料

建構程序開始後，請在 Cloud Build 資訊主頁查看進度。

1. 在 Cloud Console 中開啟 [Cloud Build](#)
 2. 按一下建構作業即可查看內容
-

步驟 4 · 約 0 分鐘

4. Artifact Registry 管理容器

建立 Artifact Registry 存放區

在本實驗室中，您將使用 Artifact Registry 儲存及掃描映像檔。使用下列指令建立存放區。

```
gcloud artifacts repositories create artifact-scanning-repo \  
  --repository-format=docker \  
  --location=us-central1 \  
  --description="Docker repository"
```

設定 Docker，在存取 Artifact Registry 時使用 gcloud 憑證。

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

更新 Cloud Build 管道

修改建構管道，將產生的映像檔推送至 Artifact Registry

```
cat > ./cloudbuild.yaml << EOF  
steps:  
  
# build  
- id: "build"  
  name: 'gcr.io/cloud-builders/docker'  
  args: ['build', '-t', 'us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-  
repo/sample-image', '.']  
  waitFor: ['-']  
  
# push to artifact registry  
- id: "push"  
  name: 'gcr.io/cloud-builders/docker'  
  args: ['push', 'us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-  
image']  
  
images:  
  - us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image  
EOF
```

執行 CI 管道

提交建構作業以供處理

```
gcloud builds submit
```


5. 自動掃描安全漏洞

每當您將新映像檔推送到 Artifact Registry 或 Container Registry 時，系統就會自動觸發 Artifact 掃描作業。發現新的安全漏洞時，系統會持續更新安全漏洞資訊。在本節中，您將查看剛建構並推送至 Artifact Registry 的映像檔，並探索安全漏洞結果。

查看映像檔詳細資料

上一個建構程序完成後，請在 Artifact Registry 資訊主頁查看映像檔和安全漏洞結果。

1. 在 Cloud 控制台開啟 [Artifact Registry](#)
2. 點選「artifact-scanning-repo」查看內容
3. 點選圖片詳細資料
4. 點選映像檔最新的摘要
5. 掃描完成後，點選映像檔的安全漏洞分頁標籤

在「安全漏洞」分頁，您會看到新建映像檔的自動掃描結果。

Google Cloud

qwiklabs-gcp-04-986b59da1f76

Search (/) for resources, docs, products, and more

Search

Artifact Registry

Repositories

Settings

350d1a869d1ea

DELETE

SETUP INSTRUCTIONS

DEPLOY

REFRESH

SHOW ALL VERSIONS

us-central1-docker.pkg.dev

artifact-scanning-repo

sample-image

sha256:350d1a869d1ea96cd930417d13be6a0d9be96ab1206327a26c06034f5f192bdf

OVERVIEW

VULNERABILITIES

PULL

MANIFEST

Scan results

Based on factors such as exploitability, scope, impact, and maturity of the vulnerability.

Scans	Total	Fixes	Critical	High	Medium
3	368	60	6	35	101

Filter Filter vulnerabilities

Name	Effective severity	CVSS	Fix available	Package	Package type	
CVE-2017-16997	Critical	9.3	Yes	glibc	OS	VIEW FIX
CVE-2015-20107	Critical	8	–	python3.5	OS	VIEW
CVE-2022-2068	Critical	10	–	openssl	OS	VIEW
CVE-2019-3462	Critical	9.3	Yes	apt	OS	VIEW FIX
CVE-2022-1292	Critical	10	Yes	openssl	OS	VIEW FIX
CVE-2019-19814	Critical	9.3	–	linux	OS	VIEW
CVE-2018-12931	High	7.2	–	linux	OS	VIEW
CVE-2019-0145	High	7.2	–	linux	OS	VIEW
CVE-2018-1000001	High	7.2	–	glibc	OS	VIEW
CVE-2017-18269	High	7.5	Yes	glibc	OS	VIEW FIX
CVE-2018-15686	High	7.2	Yes	systemd	OS	VIEW FIX
CVE-2022-1271	High	8.8	Yes	gzip	OS	VIEW FIX

自動掃描功能預設為啟用。請參閱 [Artifact Registry 設定](#)，瞭解如何開啟/關閉自動掃描功能。

6. On Demand Scanning

可能會有各種情況會要求您先執行掃描，才能將映像檔推送至存放區。舉例來說，容器開發人員可能會掃描映像檔並修正問題，然後再將程式碼推送至原始碼控管。下方的範例中，您將在本機建構及分析映像檔，然後根據結果採取行動。

建構映像檔

在這個步驟中，您會使用本機 Docker 將映像檔建構至本機快取。

```
docker build -t us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image .
```

掃描圖片

建構映像檔後，請要求掃描映像檔。掃描結果會儲存在中繼資料伺服器。工作完成後，中繼資料伺服器會提供結果的位置。

```
gcloud artifacts docker images scan \
  us-central1-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image \
  --format="value(response.scan)" > scan_id.txt
```

查看輸出檔案

請花點時間查看上一個步驟的輸出內容，這些內容會儲存在 scan_id.txt 檔案中。請注意中繼資料伺服器中掃描結果的報表位置。

```
cat scan_id.txt
```

查看詳細掃描結果

如要查看實際的掃描結果，請在輸出檔案中記下的報表位置使用 `list-vulnerabilities` 指令。

```
gcloud artifacts docker images list-vulnerabilities $(cat scan_id.txt)
```

輸出內容包含大量有關映像檔中所有安全漏洞的資料。

標記重大問題

人類很少直接使用儲存在報表的資料，通常是自動化程序使用這些結果。使用下列指令讀取報表詳細資料，並記錄是否發現任何重大安全漏洞

```
export SEVERITY=CRITICAL

gcloud artifacts docker images list-vulnerabilities $(cat scan_id.txt) --
format="value(vulnerability.effectiveSeverity)" | if grep -Fxq ${SEVERITY}; then echo "Failed
vulnerability check for ${SEVERITY} level"; else echo "No ${SEVERITY} Vulnerabilities found";
fi
```

這項指令的輸出內容如下

```
Failed vulnerability check for CRITICAL level
```

步驟 7 · 約 0 分鐘

7. 在 Cloud Build 的 CICD 中掃描

提供 Cloud Build 服務帳戶的存取權

Cloud Build 必須具備 On-Demand Scanning API 的存取權。使用下列指令提供存取權。

```
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/iam.serviceAccountUser"

gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/ondemandscanning.admin"
```

更新 Cloud Build 管道

下列指令會在目錄中建立 cloudbuild.yaml 檔案，供自動化程序使用。本範例的步驟僅限於容器建構程序。不過，實際上，除了容器步驟，您還會加入應用程式專用操作說明和測試。

使用下列指令建立檔案。

```

cat > ./cloudbuild.yaml << EOF
steps:

# build
- id: "build"
  name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-
repo/sample-image', '.']
  waitFor: ['-']

#Run a vulnerability scan at _SECURITY level
- id: scan
  name: 'gcr.io/cloud-builders/gcloud'
  entrypoint: 'bash'
  args:
    - '-c'
    - |
      (gcloud artifacts docker images scan \
      us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image \
      --location us \
      --format="value(response.scan)") > /workspace/scan_id.txt

#Analyze the result of the scan
- id: severity check
  name: 'gcr.io/cloud-builders/gcloud'
  entrypoint: 'bash'
  args:
    - '-c'
    - |
      gcloud artifacts docker images list-vulnerabilities \$(cat /workspace/scan_id.txt) \
      --format="value(vulnerability.effectiveSeverity)" | if grep -Fxq CRITICAL; \
      then echo "Failed vulnerability check for CRITICAL level" && exit 1; else echo "No
CRITICAL vulnerability found, congrats !" && exit 0; fi

#Retag
- id: "retag"
  name: 'gcr.io/cloud-builders/docker'
  args: ['tag', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-
image', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image:good']

#pushing to artifact registry
- id: "push"
  name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-
image:good']

images:
  - us-centrall-docker.pkg.dev/${PROJECT_ID}/artifact-scanning-repo/sample-image
EOF

```

執行 CI 管道

提交建構作業進行處理，確認在發現重大嚴重性安全漏洞時，是否會建構失敗。

```
gcloud builds submit
```

查看建構失敗情形

由於映像檔含有重大安全漏洞，您剛提交的建構作業將會失敗。

在「[Cloud Build 記錄](#)」頁面中查看建構失敗情形

修正安全漏洞

更新 Dockerfile，使用不含重大安全漏洞的基本映像檔。

使用下列指令覆寫 Dockerfile，改用 Debian 10 映像檔：

```
cat > ./Dockerfile << EOF
from python:3.8-slim

# App
WORKDIR /app
COPY . ./

RUN pip3 install Flask==2.1.0
RUN pip3 install gunicorn==20.1.0

CMD exec gunicorn --bind :$PORT --workers 1 --threads 8 main:app

EOF
```

使用良好圖片執行 CI 程序

提交建構作業進行處理，確認在未發現重大嚴重性安全漏洞時，是否會建構成功。

```
gcloud builds submit
```

查看建構作業是否成功

您剛提交的建構作業會成功，因為更新後的映像檔不含重大安全漏洞。

在「[Cloud Build 記錄](#)」頁面中查看建構作業是否成功

查看掃描結果

在 Artifact Registry 查看正常映像檔

1. 在 Cloud 控制台開啟 [Artifact Registry](#)
2. 點選「artifact-scanning-repo」查看內容
3. 點選圖片詳細資料

- 4. 點選映像檔最新的摘要
 - 5. 點選映像檔的安全漏洞分頁標籤
-

步驟 8 · 約 0 分鐘

8. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容：

- 使用 Cloud Build 建構映像檔
- Artifact Registry 管理容器
- 自動掃描安全漏洞
- On Demand Scanning
- 在 Cloud Build 的 CICD 中掃描

後續步驟：

- [保護部署至 Cloud Run 和 Google Kubernetes Engine 的映像檔 | Cloud Build 說明文件](#)
- [快速入門導覽課程：為 GKE | Google Cloud 設定二進位授權政策](#)

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。

—

上次更新時間：2023 年 3 月 21 日
